Indexers

What are Indexers

- Allows an object to be indexed such as an array.
- Instance of this class can be accessed by using array access operator []
- Use get and set accessors fro defining an indexer
- Index can only be defined by using **this** keyword.

Example of Indexer in .NET

To retrieve data from a specific column when looping through **SqlDataReader** object, we use either integral indexer or string indexer.

```
string CS = ConfigurationManager.ConnectionStrings["DBCS"].ConnectionString;
using (SqlConnection con = new SqlConnection(CS))
{
    SqlCommand cmd = new SqlCommand("Select * from tblEmployee", con);
    con.Open();
    SqlDataReader rdr = cmd.ExecuteReader();
    while (rdr.Read())
    {
        // Using integral indexer to retrieve Id column value
        Response.Write("Id = " + rdr[0].ToString() + " ");
        // Using string indexer to retrieve Id column value
        Response.Write("Name = " + rdr["Name"].ToString());
        Response.Write("<br/>");
    }
}
```

```
public class SqlDataReader : DbDataReader, IDataReader
{
    protected SqlConnection Connection { get; }
    public override int Depth { get; }
    public override int FieldCount { get; }
    public override bool HasRows { get; }
    public override bool IsClosed { get; }
    public override int RecordsAffected { get; }
    public override int VisibleFieldCount { get; }

    public override object this[int i] { get; }
    public override object this[string name] { get; }
```

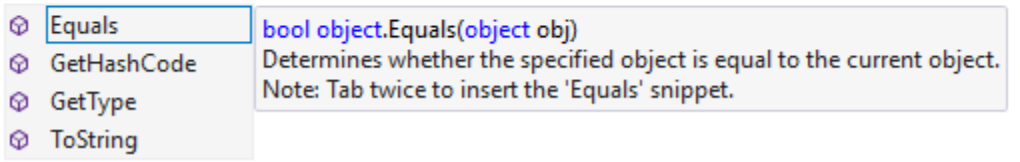
Building Indexers

- Lets understand Indexers in C# using a simple example. We have an Employee class with private members and a public constructor.

```
class Employee
{
    int Id { get; set; }
    string Name { get; set; }
    int Salary { get; set; }

    public Employee(int Id, string Name, int Salary)
    {
        this.Id = Id;
        this.Name = Name;
        this.Salary = Salary;
    }
}
```

```
class Program
{
    static void Main(string[] args)
    {
        Employee emp = new Employee(100, "ABC", 1000);
        emp.
    }
}
```



Note here that emp object doesnot have access to its private data members outside the class. This is the intended behavior. We can access the data members of a class outside the class using the following 3 options:

1. Making the fields public. (bad idea)
2. Using properties with get, set accessors
3. Using Indexers.

Building Indexers

```
class Employee
{
    int Id { get; set; }
    string Name { get; set; }
    int Salary { get; set; }

    public Employee(int Id, string Name, int Salary)
    {
        this.Id = Id;
        this.Name = Name;
        this.Salary = Salary;
    }
}
```

```
class Program
{
    static void Main(string[] args)
    {
        Employee emp = new Employee(100, "ABC", 1000);
        emp.
    }
}
```

Equals
GetHashCode
GetType
ToString

bool object.Equals(object obj)
Determines whether the specified object is equal to the current object.
Note: Tab twice to insert the 'Equals' snippet.

- Indexers allows an object to be indexed such as an array. i.e.
  - emp[0] will retrieve the first data member of emp i.e. Id
  - emp[1] will retrieve the second data member of emp i.e. Name
  - emp[2] will retrieve the third data member of emp i.e. Salary

Building Indexers

- **Syntax**:

  [<modifier>] <type> this [int index]
  {
  [get{statements}] //get accessor
  [set{statements}] //set accessor
  }

- For our case the modifier will be public.
- <type> - lets see the types of data members that will be returned –

  int & string

  So, our return type should be **object** as both int and string inherits from ultimate base class Object. It is a best practice to return **object.**
- this – this keyword is used to tell that we are defining a indexer on the current accessed object's class.

Building Indexers

Here we have defined an indexer using index.

Note: **value** is object type. So we need to explicitly cast it.

```
static void Main(string[] args)
{
    Employee emp = new Employee(100, "ABC", 1000);

    Console.WriteLine("Id: {0}", emp[0]);
    Console.WriteLine("Name: {0}", emp[1]);
    Console.WriteLine("Salary: {0}", emp[2]);

    emp[1] = "XYZ";
    Console.WriteLine("Name: {0}", emp[1]);
    Console.ReadLine();
}
```

```
class Employee{

    int Id { get; set; }
    string Name { get; set; }
    int Salary { get; set; }

    public Employee(int Id, string Name, int Salary)
    {
        this.Id = Id;
        this.Name = Name;
        this.Salary = Salary;
    }

    //indexer
    public object this[int index]
    {
        get{
            if (index == 0)
                return Id;
            else if (index == 1)
                return Name;
            else if (index == 2)
                return Salary;
            return null;
        }
        set
        {
            if (index == 0)
                Id = (int)value;
            else if (index == 1)
                Name = (string)value;
            else if (index == 2)
                Salary = (int)value;
        }
    }
}
```

Building Indexers

We can also have indexers based on string

```
public object this[string key]
{
    get
    {
        if (key == "Id")
            return Id;
        else if (key == "Name")
            return Name;
        else if (key == "Salary")
            return Salary;
        return null;
    }
    set
    {
        if (key == "Id")
            Id = (int)value;
        else if (key == "Name")
            Name = (string)value;
        else if (key == "Salary")
            Salary = (int)value;
    }
}
```

```
static void Main(string[] args)
{
    Employee emp = new Employee(100, "ABC", 1000);

    Console.WriteLine("Id: {0}", emp[0]);
    Console.WriteLine("Name: {0}", emp[1]);
    Console.WriteLine("Salary: {0}", emp[2]);

    Console.WriteLine();
    emp["Name"] = "XYZ";

    Console.WriteLine("Id: {0}", emp[0]);
    Console.WriteLine("Name: {0}", emp[1]);
    Console.WriteLine("Salary: {0}", emp[2]);
    Console.ReadLine();
}
```

```
Id: 100
Name: ABC
Salary: 1000

Id: 100
Name: XYZ
Salary: 1000
```

